Module 13 — Graph Representation & Traversal

Sources: CLRS 4e ch. 20 (Elementary Graph Algorithms) · Skiena 3e ch. 7 (Graph Traversal)

Big Idea

> *"Graphs are one of the unifying themes of computer science — an abstract representation that describes the organization of transportation systems, human interactions, and telecommunication networks. That so many different structures can be modeled using a single formalism is a source of great power to the educated programmer."*

And the sentence that should govern how you use this module:

> ***"Designing truly novel graph algorithms is a difficult task, but usually unnecessary. The key to using graph algorithms effectively in applications lies in correctly modeling your problem so you can take advantage of existing algorithms."*** *— Skiena*

Everything in this module is one of two things: how to *store* a graph, and how to *walk* it. There are exactly two walks — **BFS** (queue) and **DFS** (stack) — and they differ in one line of code: *what container holds the discovered-but-not-processed vertices.* From those two walks, at linear cost, you get shortest paths in unweighted graphs, connected components, bipartiteness, cycle detection, articulation vertices, bridges, topological order, and strongly connected components.

The reason both are `Θ(V + E)` and not more is the traversal invariant: **mark each vertex when you first see it, and never expand it twice.** Each undirected edge is examined exactly twice (once from each endpoint); each directed edge exactly once.

The deeper idea, which is CLRS's contribution: DFS is not just "a way to visit everything". It **imposes structure**. Discovery/finish timestamps nest like parentheses; every edge falls into one of four classes; and those two facts alone give you topological sort, cycle detection, SCCs, and biconnectivity — each in a few extra lines on top of the same traversal.

Remember months later: *adjacency lists, always. BFS = queue = shortest paths in unweighted graphs. DFS = stack/recursion = timestamps + edge classification, and that structure is where all the clever algorithms come from. Both* $\Theta(V+E)$.

What You Should Be Able To Do After This Chapter

- Choose between adjacency lists and matrices from the graph's density, and justify the choice with the size trade-off.
- Write BFS and DFS from memory in under three minutes each, including the parent array.
- Prove BFS computes shortest-path distances (Lemmas 20.1–20.3, Theorem 20.5).
- State and use the **parenthesis theorem** and the **white-path theorem**.
- Classify every edge as tree / back / forward / cross from vertex colors, and know which classes can appear in undirected graphs.
- Detect cycles in both undirected and directed graphs, and say precisely why the tests differ.
- Compute articulation vertices and bridges in one DFS with `low[]` values, and explain the three cases.
- Topologically sort with DFS and with Kahn's algorithm, and prove the DFS version correct.
- Compute strongly connected components with Kosaraju's two-pass algorithm, and explain why the second pass on G^T in decreasing finish order works.
- Recognize which of these six or seven traversal applications a given interview problem actually is.

Part 1 — Modeling: The Flavors of Graphs (Skiena 7.1)

`G = (V, E)`: a set of vertices with a set of vertex pairs. **The first step in any graph problem is determining the flavor of graph you're dealing with**, because it changes both the representation and the available algorithms.

DISTINCTION	MEANING	CONSEQUENCE
Undirected vs. directed	`(x,y) ∈ E ⟹ (y,x) ∈ E`?	Directed graphs get forward and cross edges in DFS; SCC ≠ connected components
Weighted vs. unweighted	numeric value on edges/vertices	*"For unweighted graphs, a shortest path is one that has the fewest number of edges, and can be found using BFS. Shortest paths in weighted graphs requires more sophisticated algorithms"* (M15)
Simple vs. non-simple	no self-loops `(x,x)`, no multiedges	*"any graph that avoids them is called simple"* — Skiena's implementations, and most textbook algorithms, assume simple
Sparse vs. dense	`|E| ≈ |V|` vs. `|E| ≈ |V|²`	Decides list vs. matrix. Complete simple undirected graph has `C(n,2) = (n²−n)/2` edges
Cyclic vs. acyclic	contains a cycle?	**DAGs** support topological sort; trees are connected acyclic undirected graphs
Embedded vs. topological	do vertices have geometric positions?	Grids and Euclidean TSP have edges *implied by* geometry
Implicit vs. explicit	is the graph built as you traverse it?	Backtracking search, web crawling — *"It is often easier to work with an implicit graph than to explicitly construct and store the entire thing"*
Labeled vs. unlabeled	do vertex identities matter?	Isomorphism testing; most network-science properties are label-invariant

Why sparse is the normal case: *"Sparse graphs are usually sparse for application-specific reasons. Road networks must be sparse because of the complexity of road junctions. The ghastliest intersection I have ever managed to identify is the endpoint of just seven different roads."*

The friendship-graph exercise (Skiena 7.1.1) is a genuinely good way to fix the vocabulary. *"'Talking the talk' proves to be an important part of 'walking the walk'":*

- *If I am your friend, does that mean you are my friend?* → **directed vs. undirected**. The "heard-of" graph is directed (*"I have heard of many famous people who have never heard of me"*).
- *How close a friend are you?* → **weighted**.

- *Am I my own friend?* → **simple** (self-loops, multiedges).
- *Who has the most friends?* → **degree**. *"Remote hermits are associated with degree-zero vertices."*
- *Do my friends live near me?* → **embedded**.
- *Oh, you also know her?* → **implicit** vs. explicit. *"The 'six degrees of separation' theory argues that there is a short path linking every two people in the world… but offers us no help in actually finding this path."*
- *Are you truly an individual?* → **labeled** vs. unlabeled.

> ***Take-Home Lesson (Skiena):*** *"Graphs can be used to model a wide variety of structures and relationships. Graph-theoretic terminology gives us a language to talk about them."*

Part 2 — Representations (CLRS 20.1, Skiena 7.2)

Adjacency list: an array `Adj[1..|V|]` of lists; `Adj[u]` holds every `v` with `(u,v) ∈ E`.

- Directed: total list length `= |E|`. Undirected: `= 2|E|` (each edge appears in both endpoints' lists).
- Memory `Θ(V + E)`. Enumerating all edges takes `Θ(V + E)` — the `Θ(V)` is unavoidable because you must look at every list, even the empty ones.
- Weights: just store `w(u,v)` alongside `v` in `u`'s list. *"The adjacency-list representation is quite robust in that you can modify it to support many other graph variants."*
- **The one weakness:** no fast "is `(u,v)` an edge?" — you must scan `Adj[u]`.

Adjacency matrix: `|V| × |V|` matrix with `a_ij = 1` iff `(i,j) ∈ E`.

- Memory `Θ(V²)` **regardless of edge count**; enumerating all edges takes `Θ(V²)`.
- Undirected ⟹ `A = Aᵀ`, so you can store only the upper triangle and halve the memory.
- Unweighted ⟹ **one bit per entry**, which is a real constant-factor advantage for small dense graphs.

Skiena's comparison table — worth memorizing wholesale:

COMPARISON	WINNER
Faster to test if `(x,y)` is in the graph?	adjacency matrices
Faster to find the degree of a vertex?	adjacency lists
Less memory on sparse graphs?	adjacency lists — `(m + n)` vs. `n²`
Less memory on dense graphs?	adjacency matrices (a small win)
Edge insertion or deletion?	adjacency matrices, `O(1)` vs. `O(d)`
Faster to traverse the graph?	adjacency lists — `θ(m + n)` vs. `θ(n²)`
Better for most problems?	**adjacency lists**

The Manhattan calculation, which makes the point concrete: a street map of Manhattan has ~15 avenues × ~200 streets ≈ **3 000 vertices and 6 000 edges** (nearly every junction has degree 4, and each edge is shared by two vertices). An adjacency matrix would be `3 000 × 3 000 = 9 000 000 entries, almost all empty`.

> ***Take-Home Lesson (Skiena):*** *"Adjacency lists are the right data structure for most applications of graphs."*
>
> ***And the caveat about the matrix's apparent weakness:*** *"Adjacency lists make it harder to verify whether a given edge `(i,j)` is in `G`... However, it is surprisingly easy to design graph algorithms that avoid any need for such queries. Typically, we sweep through all the edges of the graph in one pass via a breadth-first or depth-first traversal."*

War Story: I Was a Victim of Moore's Law (Skiena 7.3)

Skiena wrote **Combinatorica**, a Mathematica graph library, in 1990. He chose **adjacency matrices**, reasoning that for very small graphs the space advantage of lists (`n + 2m` words vs. `n²`) only kicks in when `n + 2m ≪ n²`, and Mathematica handled regular structures better than irregular ones. Two complex problems on **9- and 16-vertex** graphs took *minutes*.

Years passed. Users started complaining. *"The mystery wasn't that my program was slow, because it had always been slow. The question was why did it take them so many years to figure this out?"* **Because hardware got faster, so people tried bigger graphs — and the quadratic data structure stopped scaling.** A rewrite to edge lists let Combinatorica handle graphs **50–100× larger**.

Three lessons, all worth keeping:

1. **"To make a program run faster, just wait"** — 15 years of hardware evolution

gave a >200× speedup for free.

2. **"Asymptotics eventually do matter"** — *"future computers will have more memory and run faster than today's. This gives the edge to asymptotically more efficient algorithms/data structures, even if their performance is close on today's instances. If the implementation complexity is not substantially greater, play it safe and go with the better algorithm."*
3. **"Constant factors can matter"** — moving the structures into compiled Mathematica core gave another 10×. *"Speeding up a computation by a factor of 10 is often very important, taking it from a week down to a day, or a year down to a month."*

*(There's also a nice detail: a bump in the running-time curve at `n ≈ 250`, which Skiena attributes to a memory-hierarchy transition. **"Cache performance in data structure design should be an important but not overriding consideration. The asymptotic gains due to adjacency lists more than trumped any impact of the cache."**)*

War Story: Getting the Graph (Skiena 7.4)

A student took **five minutes** just to build the dual graph of a few-thousand-triangle mesh. Her method: for each new triangle, compare it against every previously read triangle to see whether they share two vertices — `Θ(n²)`.

The fix: an array indexed by *original mesh vertex*, each holding the list of triangles through that vertex. By Euler's formula, the average point is in fewer than six triangles, so each new triangle is compared against **fewer than twenty** others instead of thousands. Build time dropped to seconds.

> ***Take-Home Lesson:*** *"Even elementary problems like initializing data structures can prove to be bottlenecks in algorithm development. Programs working with large amounts of data must run in linear or near-linear time. Such tight performance demands leave no room to be sloppy."*

C++ Implementation — the graph type used throughout this module

```cpp
#include <vector>

class Graph {
public:
    Graph(int n, bool directed) : adj_(n), directed_(directed) {}

    void addEdge(int u, int v) {
```

```
        adj_[u].push_back(v);
        if (!directed_) adj_[v].push_back(u);
        ++edges_;
    }

    int n() const { return (int)adj_.size(); }
    int m() const { return edges_; }
    bool directed() const { return directed_; }
    const vector<int>& adj(int u) const { return adj_[u]; }

    Graph transpose() const {                       // G^T: all
edges reversed
        Graph t(n(), true);
        for (int u = 0; u < n(); ++u)
            for (int v : adj_[u]) t.addEdge(v, u);
        return t;
    }

    vector<vector<char>> toMatrix() const {
        vector<vector<char>> a(n(), vector<char>(n(), 0));
        for (int u = 0; u < n(); ++u) for (int v : adj_[u]) a[u][v]
= 1;
        return a;
    }

private:
    vector<vector<int>> adj_;
    bool directed_;
    int edges_ = 0;
};
```

Implementation notes. `std::vector<std::vector<int>>` rather than Skiena's hand-rolled linked lists: same asymptotics, far better cache behavior, no manual memory management. For very large graphs the standard competitive-programming refinement is **CSR** (compressed sparse row): one `head[]` array of offsets plus one flat `to[]` array — a single allocation, perfect locality. `transpose()` is `Θ(V + E)`, which is Exercise 20.1-3.

Part 3 — The Traversal Skeleton (Skiena 7.5)

Both searches are the same algorithm with a different container. The key idea: **mark each vertex when you first visit it, and keep track of what you have not yet completely explored.**

Each vertex is in one of three states:

STATE	MEANING	CLRS COLOR
Undiscovered	initial, untouched	WHITE
Discovered	found, but not all its edges checked yet — *on the frontier*	GRAY
Processed	all its incident edges have been examined	BLACK

`undiscovered → discovered → processed`, always in that order, never backwards.

To explore `v`, examine every edge leaving `v`. If it goes to an **undiscovered** vertex `x`, mark `x` discovered and add it to the work list. If it goes to a **processed** vertex, ignore it — *"further contemplation will tell us nothing new about the graph."* If it goes to a **discovered but not processed** vertex, ignore it too — that destination is already on the list.

Why the traversal is complete: *"Suppose that there exists a vertex `u` that remains unvisited, whose neighbor `v` was visited. This neighbor `v` will eventually be explored, after which we will certainly visit `u`."*

Edge accounting: each undirected edge is considered exactly **twice** (once from each endpoint); each directed edge exactly **once**.

And the one line that separates the two algorithms:

CONTAINER	EXPANSION ORDER	SEARCH
Queue (FIFO)	oldest unexplored vertex first — *"explorations radiate out slowly from the starting vertex"*	**BFS**
Stack (LIFO)	newest first — *"forging steadily along a path… backing up only when we are surrounded by previously discovered vertices"*	**DFS**

Part 4 — Breadth-First Search (CLRS 20.2, Skiena 7.6–7.7)

The algorithm

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE ;  u.d = ∞ ;  u.π = NIL
 5  s.color = GRAY ;  s.d = 0 ;  s.π = NIL
 8  Q = ∅ ;  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each vertex v in G.Adj[u]
13          if v.color == WHITE
14              v.color = GRAY ;  v.d = u.d + 1 ;  v.π = u
17              ENQUEUE(Q, v)
18      u.color = BLACK
```

→ **C++ implementation:** A1 BFS

Loop invariant: *at the test in line 10, the queue `Q` consists of exactly the set of gray vertices.* Every vertex painted gray is enqueued; every vertex dequeued is painted black.

"You can think of it as discovering vertices in waves emanating from the source vertex" — first all vertices at distance 1, then 2, and so on. The queue *"contains portions of two consecutive waves at any time."*

Running time `O(V + E)`, by aggregate analysis (M09): each vertex is enqueued and dequeued at most once (`O(V)` queue operations), and each adjacency list is scanned exactly once when its vertex is dequeued — total scan cost `Θ(E)`. Initialization is `O(V)`.

Correctness

Let `δ(s,v)` be the true shortest-path distance (fewest edges).

Lemma 20.1. For any edge `(u,v) ∈ E`: `δ(s,v) ≤ δ(s,u) + 1`.
Proof. If `u` is reachable, so is `v`, and the shortest path to `v` can't be longer than the shortest path to `u` plus the edge. If `u` isn't reachable, `δ(s,u) = ∞`. ∎

Lemma 20.2. At all times, `v.d ≥ δ(s,v)`.
Proof. Induction on the number of `ENQUEUE` operations. Base: `s.d = 0 = δ(s,s)` and everything else is `∞`. Step: when white `v` is discovered from `u`, `v.d = u.d + 1 ≥ δ(s,u) + 1 ≥ δ(s,v)` by the inductive hypothesis and Lemma 20.1. `v` is then grayed, so it is never enqueued again and `v.d` never changes. ∎

Lemma 20.3 (the queue is nearly sorted). If $Q = \langle v_1, \ldots, v_r \rangle$ with v_1 at the head, then $v_r.d \le v_1.d + 1$ and $v_i.d \le v_{i+1}.d$. *In other words the d values in the queue always look like $\langle k, k, \ldots, k, k+1, \ldots, k+1 \rangle$.*

Corollary 20.4. If v_i is enqueued before v_j, then $v_i.d \le v_j.d$. **Distances assigned at enqueue time monotonically increase.**

Theorem 20.5 (Correctness of BFS). BFS discovers every vertex reachable from s, and at termination $v.d = \delta(s,v)$ for all v. Moreover, for any reachable $v \neq s$, a shortest path to v is a shortest path to $v.\pi$ followed by the edge $(v.\pi, v)$.

Proof. Suppose not; among the offenders pick v minimizing $\delta(s,v)$. By Lemma 20.2, $v.d > \delta(s,v)$. Let u be v's predecessor on a shortest path, so $\delta(s,v) = \delta(s,u) + 1$ and, by minimality of v, $u.d = \delta(s,u)$. Hence

$$v.d > \delta(s,v) = \delta(s,u) + 1 = u.d + 1 \tag{20.1}$$

Now look at the moment u is dequeued. v is white, gray, or black. **White** $\Rightarrow$ line 15 sets $v.d = u.d + 1$, contradicting (20.1). **Black** $\Rightarrow$ v was dequeued earlier, so by Corollary 20.4 $v.d \le u.d$, contradiction. **Gray** $\Rightarrow$ v was grayed when some earlier-dequeued w was processed, so $v.d = w.d + 1 \le u.d + 1$, contradiction. ∎

Lemma 20.6. The predecessor subgraph G_π is a **breadth-first tree**: it contains exactly the reachable vertices, and its unique simple path from s to each v is a shortest path in G.

Two things to remember when using BFS for shortest paths (Skiena states them explicitly): (1) **the tree is only useful if BFS was rooted at your source**; (2) **BFS gives shortest paths only if the graph is unweighted.**

(The distances d are independent of adjacency-list ordering; the tree is not — Exercise 20.2-5.)

Applications

Connected components. BFS from an arbitrary vertex; everything discovered is in that component; repeat from any undiscovered vertex. *"An amazing number of seemingly complicated problems reduce to finding or counting connected components. For example, deciding whether a puzzle such as Rubik's cube or the 15-puzzle can be solved from any position is really asking whether the graph of possible configurations is connected."*

Two-coloring / bipartiteness. *"We can augment breadth-first search so that whenever we discover a new vertex, we color it the opposite of its parent. We check whether any non-tree edge links two vertices of the same color. Such a conflict means that the graph cannot be two-colored."* A graph is bipartite **iff** it has no odd cycle, and a same-color non-tree edge is exactly an odd cycle. *(Note that you can pick the first vertex's color arbitrarily in each component — the graph structure alone can't tell you which side is which.)*

C++ Implementation

```
#include <algorithm>
#include <queue>
#include <vector>

struct BfsResult {
    vector<int> dist;                           // -1 == unreachable
(infinity)
    vector<int> parent;                         // -1 == none
};

BfsResult bfs(const Graph& g, int s) {
    BfsResult r{vector<int>(g.n(), -1), vector<int>(g.n(), -1)};
    queue<int> q;
    r.dist[s] = 0;
    q.push(s);
    while (!q.empty()) {
        const int u = q.front(); q.pop();
        for (int v : g.adj(u))
            if (r.dist[v] < 0) {                    // v is WHITE:
discover it
                r.dist[v] = r.dist[u] + 1;
                r.parent[v] = u;
                q.push(v);
            }
    }
    return r;
}

// PRINT-PATH: the tree path from s to v is a shortest path in G.
vector<int> bfsPath(const BfsResult& r, int s, int v) {
    if (r.dist[v] < 0) return {};
    vector<int> path;
    for (int x = v; x != -1; x = r.parent[x]) path.push_back(x);
```

```
    reverse(path.begin(), path.end());
    return path.front() == s ? path : vector<int>{};
}

// Connected components of an undirected graph, one BFS per
component.
vector<int> connectedComponents(const Graph& g, int& count) {
    vector<int> comp(g.n(), -1);
    count = 0;
    for (int s = 0; s < g.n(); ++s) {
        if (comp[s] >= 0) continue;
        queue<int> q;
        comp[s] = count;
        q.push(s);
        while (!q.empty()) {
            const int u = q.front(); q.pop();
            for (int v : g.adj(u)) if (comp[v] < 0) { comp[v] =
count; q.push(v); }
        }
        ++count;
    }
    return comp;
}

// Two-colouring: colour each discovered vertex the opposite of its
parent.
bool twoColor(const Graph& g, vector<int>& color) {
    color.assign(g.n(), -1);
    for (int s = 0; s < g.n(); ++s) {
        if (color[s] >= 0) continue;
        color[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            const int u = q.front(); q.pop();
            for (int v : g.adj(u)) {
                if (color[v] < 0) { color[v] = 1 - color[u];
q.push(v); }
                else if (color[v] == color[u]) return false;    //
conflicting non-tree edge
            }
        }
```

```
    }
    return true;
}
```

Implementation note: `dist[v] < 0` doubles as the WHITE test, so no separate color array is needed. (Exercise 20.2-3 makes exactly this point — the gray/black distinction is pedagogical, not necessary.)

Verified: on 300 random directed and undirected graphs, BFS distances matched brute-force relaxation, and every reconstructed tree path was checked to have exactly `dist[v] + 1` vertices with all consecutive pairs genuinely adjacent. Components matched pairwise reachability; two-coloring agreed with exhaustive 2^n search on 300 graphs.

Part 5 — Depth-First Search (CLRS 20.3, Skiena 7.8)

The algorithm and its timestamps

```
DFS(G)                                  DFS-VISIT(G, u)
1  for each u ∈ G.V                     1  time = time + 1
2      u.color = WHITE                  2  u.d = time            //
discovery time
3      u.π = NIL                        3  u.color = GRAY
4  time = 0                             4  for each v in G.Adj[u]
5  for each u ∈ G.V                     5      if v.color == WHITE
6      if u.color == WHITE              6          v.π = u
7          DFS-VISIT(G, u)              7          DFS-VISIT(G, v)
                                        8  time = time + 1
                                        9  u.f = time            //
finish time
                                       10  u.color = BLACK
```

→ **C++ implementation:** A2 DFS and DFS-VISIT

Note the asymmetry with BFS: DFS loops over *all* vertices as potential sources, producing a **depth-first forest** rather than a single tree. CLRS explains why: *"breadth-first search usually serves to find shortest-path distances... from a given source. Depth-first search is often a subroutine in another algorithm."*

Timestamps are integers in $1..2|V|$, and **$u.d < u.f$ always**. u is WHITE before $u.d$, GRAY between, BLACK after. $\Theta(V + E)$ by the same aggregate argument as BFS.

Skiena's two readings of the timestamps are the intuition to keep:

- **Who is an ancestor?** *"the time interval of y must be properly nested within the interval of ancestor x"* — you can't be born before your parent, and you can't exit before your descendants have.
- **How many descendants?** *"The clock gets incremented on each vertex entry and vertex exit, so half the time difference denotes the number of descendants of v."*

The two structural theorems

Theorem 20.7 (Parenthesis theorem). For any two vertices u, v, exactly one holds:

- $[u.d, u.f]$ and $[v.d, v.f]$ are **entirely disjoint**, and neither is a descendant of the other;
- $[u.d, u.f]$ is **contained in** $[v.d, v.f]$, and u is a descendant of v;
- $[v.d, v.f]$ is contained in $[u.d, u.f]$, and v is a descendant of u.

Proof. WLOG $u.d < v.d$. If $v.d < u.f$, then v was discovered while u was gray, so v is a descendant of u; and DFS cannot return to finish u until all of v's edges are explored, so $[v.d, v.f] \subset [u.d, u.f]$. Otherwise $u.f < v.d$, giving $u.d < u.f < v.d < v.f$ — disjoint intervals, so neither was gray while the other was discovered, so neither is a descendant. ∎

If DFS printed $(u$ on discovery and $u)$ on finish, the output would be a **well-formed parenthesization**. That is literally the theorem.

Corollary 20.8. v is a proper descendant of u **iff** $u.d < v.d < v.f < u.f$.

Theorem 20.9 (White-path theorem). v is a descendant of u in the depth-first forest **iff** at time $u.d$ there is a path from u to v consisting **entirely of white vertices**.

Proof (⇐, the useful direction). Suppose a white path $u \rightsquigarrow v$ exists at time $u.d$ but v doesn't become a descendant; WLOG every other vertex on the path does. Let w be v's predecessor on the path; w is a descendant of u, so $w.f \le u.f$. Since v is discovered after u but before w finishes, $u.d < v.d < w.f \le u.f$, so by Theorem 20.7 $[v.d, v.f] \subset [u.d, u.f]$, and by Corollary 20.8 v **is** a descendant — contradiction. ∎

The white-path theorem is the workhorse. It is what proves the cycle lemma (20.11) and both SCC lemmas (20.14, and via them Theorem 20.16). When you need to argue "these vertices all end up in the same DFS subtree", this is the tool.

Edge classification

TYPE	DEFINITION	COLOR OF `v` WHEN `(u,v)` FIRST EXPLORED
Tree	`v` first discovered by exploring `(u,v)`	WHITE
Back	`v` is an **ancestor** of `u` (self-loops count)	GRAY
Forward	non-tree edge to a proper **descendant**	BLACK, `u.d < v.d`
Cross	everything else — same tree but unrelated, or different trees	BLACK, `u.d > v.d`

Why gray means back edge: *"the gray vertices always form a linear chain of descendants corresponding to the stack of active DFS-VISIT invocations… an edge that reaches another gray vertex has reached an ancestor."*

Theorem 20.10. In a DFS of an **undirected** graph, **every edge is either a tree edge or a back edge.**

Proof. Let `(u,v)` be an edge, WLOG `u.d < v.d`. Since `v` is on `u`'s adjacency list, `v` is discovered and finished while `u` is gray. If the search first explores the edge from `u` to `v`, then `v` was white (otherwise the edge would already have been explored from `v` to `u`), so it's a tree edge. If it first explores from `v` to `u`, then `u` is an ancestor of `v`, so it's a back edge. ∎

Skiena's version of the same argument, which is more memorable: *"Might we encounter a 'forward edge' `(x,y)`, directed toward a descendant vertex? No, because in this case, we would have first traversed `(x,y)` while exploring `y`, making it a back edge. Might we encounter a 'cross edge' `(x,y)`? Again no, because we would have first discovered this edge when we explored `y`, making it a tree edge."*

The subtlety that catches everyone — Skiena is unusually candid about it: *"I find that the subtlety of depth-first search-based algorithms kicks me in the head whenever I try to implement one."* (Footnote: *"Indeed, the most horrifying errors in the previous edition of this book came in this section."*)

The specific trap: in an undirected graph, each edge `(x,y)` appears in *both* adjacency lists. When you meet `(x,y)` from `x` and `y` is gray, is this the first or second time you've seen this edge? *"Careful reflection will convince you that this must be our first traversal* ***unless*** `y` ***is the immediate ancestor of*** `x` *— that is,* `(y,x)` *is a tree edge. This can be established by testing if* `y == parent[x]`*."*

Get that `parent` test wrong and every single undirected edge looks like a back edge, so every graph looks cyclic.

Exercise 20.3-5 gives the timestamp characterization, which is how you verify a classification without tracking colors:

EDGE TYPE	TIMESTAMP CONDITION
tree or forward	`u.d < v.d < v.f < u.f`
back	`v.d ≤ u.d < u.f ≤ v.f`
cross	`v.d < v.f < u.d < u.f`

C++ Implementation

```cpp
#include <algorithm>
#include <functional>
#include <utility>
#include <vector>

enum EdgeType { TREE, BACK, FORWARD, CROSS };

struct DfsResult {
    vector<int> disc, fin, parent;             // discovery time,
finish time, predecessor
    vector<int> order;                         // vertices by
increasing finish time
    vector<pair<pair<int, int>, EdgeType>> edges;
    bool hasBackEdge = false;
};

DfsResult dfs(const Graph& g) {
    const int n = g.n();
    DfsResult r{vector<int>(n, 0), vector<int>(n, 0), vector<int>
(n, -1), {}, {}, false};
```

```
    vector<int> color(n, 0);                    // 0 = white, 1 =
gray, 2 = black
    int time = 0;

    function<void(int)> visit = [&](int u) {
        r.disc[u] = ++time;
        color[u] = 1;
        for (int v : g.adj(u)) {
            if (color[v] == 0) {                    // WHITE ->
tree edge
                r.edges.push_back({{u, v}, TREE});
                r.parent[v] = u;
                visit(v);
            } else if (color[v] == 1) {             // GRAY -> back
edge
                // for undirected graphs skip the edge back to the
immediate parent
                if (g.directed() || r.parent[u] != v) {
                    r.edges.push_back({{u, v}, BACK});
                    r.hasBackEdge = true;
                }
            } else {                                // BLACK ->
forward or cross
                if (g.directed())
                    r.edges.push_back({{u, v}, r.disc[u] <
r.disc[v] ? FORWARD : CROSS});
            }
        }
        r.fin[u] = ++time;
        color[u] = 2;
        r.order.push_back(u);
    };
    for (int u = 0; u < n; ++u) if (color[u] == 0) visit(u);
    return r;
}

// Exercise 20.3-6: DFS with an explicit stack instead of
recursion.
vector<int> dfsIterativeOrder(const Graph& g) {
    const int n = g.n();
    vector<int> color(n, 0), it(n, 0), finishOrder;
    vector<int> stk;
```

```
    for (int s = 0; s < n; ++s) {
        if (color[s] != 0) continue;
        color[s] = 1;
        stk.push_back(s);
        while (!stk.empty()) {
            const int u = stk.back();
            if (it[u] < (int)g.adj(u).size()) {
                const int v = g.adj(u)[it[u]++];
                if (color[v] == 0) { color[v] = 1;
stk.push_back(v); }
            } else {
                color[u] = 2;
                finishOrder.push_back(u);
                stk.pop_back();
            }
        }
    }
    return finishOrder;
}
```

Implementation note on `dfsIterativeOrder`: the `it[]` array — *"how far into `u`'s adjacency list have I got?"* — is what makes the iterative version equivalent to the recursive one. The naive "push all neighbors at once" stack version is **not** the same algorithm: it visits vertices in a different order and gives you no correct finish times, so it cannot be used for topological sort or SCC. **If you need an iterative DFS, you need the explicit iterator.**

Verified: on 300 random graphs, `u.d < u.f` always held; every pair of intervals was disjoint or properly nested (Theorem 20.7) with nesting matching actual ancestry (Corollary 20.8); undirected graphs produced only tree and back edges (Theorem 20.10); every classified edge satisfied its timestamp characterization (Exercise 20.3-5); and the iterative DFS produced exactly the same finish order as the recursive one.

Part 6 — Undirected Applications: Cycles, Articulation Vertices, Bridges

Cycle detection

Back edges are the key. *"If there is no back edge, all edges are tree edges, and no cycle exists in a tree. But any back edge going from* `x` *to an ancestor* `y` *creates a cycle with the tree path from* `y` *to* `x`*."*

The correctness *"depends upon processing each undirected edge exactly once. Otherwise, a spurious two-vertex cycle* `(x, y, x)` *could be composed from the two traversals of any single undirected edge."* — the `parent` test again.

(Skiena also notes a practical point: stop at the first cycle. Without an early exit you'd report a new cycle for every back edge, and a complete graph has $\Theta(n^2)$ *of them.)*

Articulation vertices and bridges

An **articulation vertex** (cut-node) is a vertex whose deletion disconnects a component. *"Any graph that contains an articulation vertex is inherently fragile."* A graph with no articulation vertex is **biconnected**. A **bridge** is an edge whose deletion disconnects the graph; a graph with none is **edge-biconnected**.

Brute force is $O(n(m+n))$ — delete each vertex and re-traverse. **DFS does it in one pass,** $O(n+m)$**.**

The intuition, which is the best part of Skiena's treatment: *"If the DFS tree represented the entirety of the graph, all internal (non-leaf) nodes would be articulation vertices... But blowing up a leaf would not disconnect the tree."* Then: ***"Think of these back edges as security cables linking a vertex back to one of its ancestors. The security cable from*** `x` ***back to*** `y` ***ensures that none of the vertices on the tree path between*** `x` ***and*** `y` ***can be articulation vertices. Delete any of these vertices, and the security cable will still hold all of them to the rest of the tree."***

Define `low[v]` (Skiena calls it `reachable_ancestor[v]`) = the earliest discovery time reachable from `v`'s subtree using at most one back edge.

The three cases (Skiena's Figure 7.13 — note *they are not mutually exclusive*):

CASE	CONDITION	WHY
Root cut-node	the DFS root has ⩾ 2 children	no edge from the second child's subtree can reach the first's — otherwise it would have been in the first subtree
Bridge cut-node	`low[v] == disc[v]`	nothing in `v`'s subtree reaches above `v`, so the tree edge `(parent[v], v)` is a **bridge**; `parent[v]` is a cut-node, and so is `v` unless `v` is a leaf
Parent cut-node	`low[v] == disc[parent[v]]`	`v`'s subtree reaches its parent but no higher, so deleting the parent severs `v` — unless the parent is the root

In the standard `low`-value formulation these collapse into two tests: **`v` non-root is a cut-node iff some child `c` has `low[c] ≥ disc[v]`**, and **`(v,c)` is a bridge iff `low[c] > disc[v]`**. The strictness of that inequality is the entire difference between "cut vertex" and "cut edge."

```
#include <algorithm>
#include <functional>
#include <utility>
#include <vector>

// low[v] = earliest discovery time reachable from v's subtree via
one back edge.
struct CutResult {
    vector<int> articulation;
    vector<pair<int, int>> bridges;
};

CutResult articulationAndBridges(const Graph& g) {
    const int n = g.n();
    vector<int> disc(n, 0), low(n, 0), parent(n, -1);
    vector<char> isArt(n, 0);
    CutResult out;
    int time = 0;

    function<void(int)> visit = [&](int u) {
        disc[u] = low[u] = ++time;
        int children = 0;
        for (int v : g.adj(u)) {
            if (disc[v] == 0) {                    // tree edge
                ++children;
```

```
                parent[v] = u;
                visit(v);
                low[u] = min(low[u], low[v]);
                if (parent[u] != -1 && low[v] >= disc[u]) isArt[u]
= 1;
                if (low[v] > disc[u]) out.bridges.push_back({min(u,
v), max(u, v)});
            } else if (v != parent[u]) {            // back edge
(not to the immediate parent)
                low[u] = min(low[u], disc[v]);
            }
        }
        if (parent[u] == -1 && children > 1) isArt[u] = 1;    //
root with 2+ children
    };
    for (int u = 0; u < n; ++u) if (disc[u] == 0) visit(u);
    for (int u = 0; u < n; ++u) if (isArt[u])
out.articulation.push_back(u);
    sort(out.bridges.begin(), out.bridges.end());
    return out;
}
```

Two details that are bugs waiting to happen. (1) `low[u] = min(low[u], disc[v])` on a back edge — **`disc[v]`, not `low[v]`**; using `low[v]` is wrong because `v` is an ancestor whose `low` may reach above `u` through a path that doesn't pass through `u`'s subtree. (2) The `v != parent[u]` guard is Skiena's parent test; with **multiedges** it must become a "skip the specific edge index you came in on" test instead, or a genuine parallel edge will be misread as the return trip.

Verified: against brute-force vertex and edge deletion (recomputing component counts) on 400 random undirected graphs, plus Skiena's Figure 7.12 instance, where it reports articulation vertices **{1, 2, 5}** and bridges **{(1,8), (5,6)}** — exactly the figure.

Part 7 — Directed Graphs

Topological sort (CLRS 20.4, Skiena 7.10.1)

A **topological sort** of a DAG is a linear order of the vertices such that every edge `(u,v)` has `u` before `v`. *"Think of a topological sort of a graph as an ordering of its vertices along a horizontal line so that all directed edges go from left to right."*

(CLRS's running example is Professor Bumstead getting dressed: socks before shoes, undershorts before pants, but socks and pants in any order.)

Skiena's argument for **why it matters beyond scheduling** is the one to keep: *"Suppose we seek the shortest (or longest) path from `x` to `y` in a DAG. No vertex `v` appearing after `y` in the topological order can possibly contribute to any such path... We can appropriately process all the vertices from left to right in topological order... and know that we will have looked at everything we need before we need it."* — **Topological order is the evaluation order that makes DP on a DAG work** (M11).

```
TOPOLOGICAL-SORT(G)
1  call DFS(G) to compute finish times v.f for each vertex v
2  as each vertex is finished, insert it onto the front of a linked
list
3  return the linked list of vertices
```

→ **C++ implementation:** A3 TOPOLOGICAL-SORT

Three lines, `Θ(V + E)`.

Lemma 20.11. A directed graph is acyclic **iff** a DFS yields **no back edges**.
Proof. (⟹) A back edge `(u,v)` means `v` is an ancestor of `u`, so there's a path `v ⇝ u` plus the edge — a cycle. (⟸) If `G` has a cycle `c`, let `v` be the first vertex of `c` discovered and `(u,v)` the preceding edge in `c`. At time `v.d` the rest of `c` is a white path from `v` to `u`, so by the **white-path theorem** `u` becomes a descendant of `v`, making `(u,v)` a back edge. ∎

Theorem 20.12. `TOPOLOGICAL-SORT` is correct.
Proof. It suffices to show `v.f < u.f` for every edge `(u,v)`. When `(u,v)` is explored, `v` can't be gray (that would be a back edge, contradicting Lemma 20.11 on a DAG). If `v` is white it becomes a descendant, so `v.f < u.f`. If `v` is black, `v.f` is already set while `u.f` is not yet, so again `v.f < u.f`. ∎

Exercise 20.4-5 gives the other standard algorithm — Kahn's: repeatedly output a vertex of in-degree 0 and remove its outgoing edges. Also `O(V+E)` with a work-list of zero-in-degree vertices. **If the output has fewer than `|V|` vertices, the graph has a cycle** — which is often more convenient than DFS's back-edge test, and gives you the "which vertices are in cycles" answer for free.

Strongly connected components (CLRS 20.5, Skiena 7.10.2)

A **strongly connected component** is a maximal set C with $u \rightsquigarrow v$ and $v \rightsquigarrow u$ for all $u, v \in C$. *"Road networks had better be strongly connected: otherwise there will be places you can drive to but not drive home from without violating one-way signs."*

```
STRONGLY-CONNECTED-COMPONENTS(G)
1  call DFS(G) to compute finish times u.f for each vertex u
2  create G^T
3  call DFS(G^T), but in the main loop consider vertices in order of
decreasing u.f
4  output the vertices of each tree in the resulting depth-first
forest as a separate SCC
```

→ **C++ implementation:** A4 STRONGLY-CONNECTED-COMPONENTS

$\Theta(V + E)$. Two DFS passes and a transpose. That's it.

Why it works — the component graph. Contract each SCC to a single vertex to get G^{SCC}. The key fact:

Lemma 20.13. If C and C' are distinct SCCs and G has a path $u \rightsquigarrow u'$ with $u \in C$, $u' \in C'$, then G has **no** path $v' \rightsquigarrow v$ back. *(Otherwise u and v' would be mutually reachable and the components wouldn't be distinct.)* $\Longrightarrow$ **G^{SCC} is a DAG.**

Lemma 20.14. If $(u,v) \in E$ with $u \in C$, $v \in C'$, then $\mathbf{f(C) > f(C')}$ — where $f(C) = \max\{u.f : u \in C\}$.
Proof. Two cases on which component is discovered first. If $d(C) < d(C')$: let x be the first vertex discovered in C. At time $x.d$ everything in both components is white, and there's a white path $x \rightsquigarrow u \to v \rightsquigarrow w$ to every $w \in C'$, so by the white-path theorem all of $C \cup C'$ are descendants of x, and by Corollary 20.8, $x.f = f(C) > f(C')$. If $d(C') < d(C)$: let y be the first vertex discovered in C'; all of C' becomes descendants of y so $y.f = f(C')$. By Lemma 20.13 there's no path $C' \to C$, so no vertex of C is reachable from y, so all of C is still white at $y.f$, giving $w.f > y.f$ for all $w \in C$ and hence $f(C) > f(C')$. ∎

Corollary 20.15. If $f(C) > f(C')$, then E^T contains **no** edge from C to C'.

That corollary is the whole algorithm. The second DFS starts at the vertex with maximum `f`, which lies in the component `C` with maximum `f(C)`. By Corollary 20.15, G^T has **no edges out of `C`**, so the search from that root can't escape `C` — the tree it grows is exactly `C`. Then repeat with the next-highest finish time, whose component can only reach components already visited. **Theorem 20.16** formalizes this by induction on the trees produced.

The one-sentence intuition: *processing in decreasing finish order visits the components of `G^SCC` in topological order, and transposing makes each component a sink you cannot escape.*

Skiena adds a useful connection: *"DAGs are directed graphs where each vertex forms its own strongly connected component. On a DAG, the top vertex on the stack will be one that cannot reach any other vertex. **The bookkeeping here is identical to topological sort.**"*

C++ Implementation

```
#include <algorithm>
#include <functional>
#include <set>
#include <utility>
#include <vector>

// DFS version: reverse of the finish order. Returns {} if a cycle
exists.
vector<int> topoSortDFS(const Graph& g) {
    const auto r = dfs(g);
    if (r.hasBackEdge) return {};
    vector<int> out(r.order.rbegin(), r.order.rend());
    return out;
}

// Exercise 20.4-5, Kahn's algorithm: repeatedly remove a vertex of
in-degree 0.
vector<int> topoSortKahn(const Graph& g) {
    const int n = g.n();
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u) for (int v : g.adj(u)) ++indeg[v];
    vector<int> ready, out;
    for (int u = 0; u < n; ++u) if (indeg[u] == 0)
ready.push_back(u);
    while (!ready.empty()) {
        const int u = ready.back(); ready.pop_back();
```

```
        out.push_back(u);
        for (int v : g.adj(u)) if (--indeg[v] == 0)
ready.push_back(v);
    }
    return (int)out.size() == n ? out : vector<int>{};
}

// Exercise 20.4-2: count simple paths a -> b in a DAG, in
topological order.
long long countPathsDAG(const Graph& g, int a, int b) {
    const auto order = topoSortDFS(g);
    if (order.empty() && g.n() > 0) return -1;                  //
not a DAG
    vector<long long> ways(g.n(), 0);
    ways[a] = 1;
    for (int u : order)
        if (ways[u] > 0)
            for (int v : g.adj(u)) ways[v] += ways[u];
    return ways[b];
}

// Kosaraju / CLRS: DFS on G for finish times, then DFS on G^T in
decreasing order.
vector<int> sccKosaraju(const Graph& g, int& count) {
    const auto first = dfs(g);                                   //
first.order is by finish time
    const Graph gt = g.transpose();
    vector<int> comp(g.n(), -1);
    count = 0;
    for (auto it = first.order.rbegin(); it != first.order.rend();
++it) {
        if (comp[*it] >= 0) continue;
        vector<int> stk{*it};
        comp[*it] = count;
        while (!stk.empty()) {
            const int u = stk.back(); stk.pop_back();
            for (int v : gt.adj(u)) if (comp[v] < 0) { comp[v] =
count; stk.push_back(v); }
        }
        ++count;
    }
    return comp;
```

```
}

// Tarjan: one pass, using low-link values and a stack of "open"
vertices.
vector<int> sccTarjan(const Graph& g, int& count) {
    const int n = g.n();
    vector<int> disc(n, 0), low(n, 0), comp(n, -1), stk;
    vector<char> onStack(n, 0);
    int time = 0;
    count = 0;

    function<void(int)> visit = [&](int u) {
        disc[u] = low[u] = ++time;
        stk.push_back(u);
        onStack[u] = 1;
        for (int v : g.adj(u)) {
            if (disc[v] == 0) { visit(v); low[u] = min(low[u],
low[v]); }
            else if (onStack[v])  low[u] = min(low[u], disc[v]);
        }
        if (low[u] == disc[u]) {                                  //
u roots an SCC
            for (;;) {
                const int w = stk.back(); stk.pop_back();
                onStack[w] = 0;
                comp[w] = count;
                if (w == u) break;
            }
            ++count;
        }
    };
    for (int u = 0; u < n; ++u) if (disc[u] == 0) visit(u);
    return comp;
}

// Exercise 20.5-5: the component (condensation) graph, with no
duplicate edges.
Graph condensation(const Graph& g, const vector<int>& comp, int
count) {
    Graph c(count, true);
    set<pair<int, int>> seen;
    for (int u = 0; u < g.n(); ++u)
```

```
        for (int v : g.adj(u))
            if (comp[u] != comp[v] && seen.insert({comp[u],
comp[v]}).second)
                c.addEdge(comp[u], comp[v]);
    return c;
}
```

A property worth knowing about both SCC algorithms: Kosaraju numbers components in topological order of `G^SCC` (every condensation edge goes from a lower to a higher component number), while **Tarjan numbers them in reverse topological order** (it closes sink components first). Whichever you use, you get the condensation's topological order for free — no second sort needed. Which one you have matters when you immediately run a DP over the condensation, which is the usual next step.

Verified: on 400 random digraphs, Kosaraju and Tarjan agreed with each other and with mutual reachability computed by transitive closure; the condensation was always acyclic; and Kosaraju's numbering always satisfied `comp[u] < comp[v]` for every cross-component edge. On CLRS's Figure 20.9 instance it recovers exactly the four components **{a,b,e}, {c,d}, {f,g}, {h}**, and Professor Bumstead's dag topologically sorts with socks and undershorts before shoes and shirt before tie before jacket.

> *Outside / Engineering Context — Kosaraju vs. Tarjan vs. Kahn*
>
> - ***Tarjan's algorithm is one DFS instead of two and doesn't build*** `G^T`*, so it's usually ~2× faster and uses less memory. Use it in production and in contests.* ***Learn Kosaraju anyway*** *— it is far easier to explain in an interview, and the proof is the one CLRS gives.*
> - ***Path-based SCC (Gabow/Cheriyan–Mehlhorn)*** *is a third one-pass algorithm using two stacks, often marginally faster still.*
> - ***Kahn's topological sort is preferred in practice*** *over the DFS version for two reasons: no recursion (so no stack overflow on a million-vertex dependency graph), and it naturally yields the lexicographically smallest topological order if you use a priority queue instead of a plain work-list — which is what build systems and package managers want for reproducibility.*
> - ***2-SAT is the classic SCC application:*** *build the implication graph over* `2n` *literals, and the formula is satisfiable* ***iff*** *no variable has* `x` *and* `¬x` *in the same SCC; the satisfying assignment reads off the condensation's topological order.*

Recognition Patterns

PROBLEM SAYS	USE
"fewest moves / minimum steps / shortest path" on an **unweighted** graph or grid	**BFS**
"is there a path", "how many groups", "islands", "is the network connected"	BFS or DFS + components
"can we split into two sets with no internal conflicts"	**two-coloring** (bipartite check)
"detect a cycle" in an **undirected** graph	DFS back edge (with the parent guard), or union–find (M10)
"detect a cycle" in a **directed** graph	DFS back edge = **gray target**, or Kahn's leftover vertices
"order tasks respecting prerequisites", "course schedule", "build order"	**topological sort**
"count paths / longest path / DP" on a DAG	topological order, then relax edges left to right
"single point of failure", "critical server/cable", "biconnectivity"	**articulation vertices / bridges** with `low[]`
"mutually reachable", "everyone can reach everyone", one-way streets	**SCC**
"2-SAT", "implication constraints"	SCC on the implication graph
"shortest path with **weights**"	**not this module** — Dijkstra / Bellman-Ford (M15)

The single most common mistake at the recognition stage: reaching for Dijkstra when the graph is unweighted. **BFS is `O(V+E)`; Dijkstra is `O((V+E) lg V)` and needs a heap.** If every edge costs 1, use BFS.

The second most common: using DFS to find shortest paths. **DFS finds *a* path, essentially never the shortest one.**

Common Mistakes

MISTAKE	CONSEQUENCE
Marking a vertex visited when **dequeuing** instead of when **enqueuing** in BFS	Vertices enter the queue many times; `O(V·E)` blowup and possibly wrong distances
Forgetting the `v != parent[u]` guard in undirected DFS	Every edge looks like a back edge; every graph looks cyclic
Using `low[v]` instead of `disc[v]` when relaxing a **back** edge	Wrong articulation vertices — a classic, and it passes many test cases
`low[c] >= disc[u]` vs. `low[c] > disc[u]`	The first is the cut-vertex test, the second is the bridge test. Swapping them is silently wrong
Naive iterative DFS that pushes all neighbors at once	Different traversal order, **no valid finish times** — breaks topological sort and SCC
Recursive DFS on a graph with 10^6 vertices	Stack overflow. Use the explicit-iterator version
Running SCC's second pass on `G` instead of G^T	Wrong (Exercise 20.5-3 asks you to find the counterexample)
Sorting by **discovery** time rather than **finish** time for topological sort	Wrong order; discovery order is not a topological order
Forgetting that DFS is a **forest**, not a tree	Missing components on a disconnected or non-strongly-connected input
Treating connected components and SCCs as the same thing on a digraph	Weakly connected ≠ strongly connected
Building an adjacency matrix for a sparse graph	Manhattan: 9 000 000 entries for 6 000 edges
Computing the dual/derived graph by pairwise comparison	Skiena's war story: $\Theta(n^2)$ where $\Theta(n)$ is available

---

Complexity Summary

OPERATION	ADJACENCY LIST	ADJACENCY MATRIX
Memory	$\Theta(V + E)$	$\Theta(V^2)$
Is (u,v) an edge?	$O(\deg u)$	$\Theta(1)$
Enumerate u's neighbors	$\Theta(\deg u)$	$\Theta(V)$
Enumerate all edges	$\Theta(V + E)$	$\Theta(V^2)$
Insert / delete an edge	$O(1)$ insert, $O(\deg u)$ delete	$\Theta(1)$
BFS / DFS	$\Theta(V + E)$	$\Theta(V^2)$

ALGORITHM	TIME	SPACE	OUTPUT
BFS	$\Theta(V + E)$	$\Theta(V)$	distances d, BFS tree π
DFS	$\Theta(V + E)$	$\Theta(V)$ (+ $O(V)$ stack)	timestamps d/f, DFS forest, edge classes
Connected components	$\Theta(V + E)$	$\Theta(V)$	component id per vertex
Two-coloring / bipartite	$\Theta(V + E)$	$\Theta(V)$	2-coloring or a conflict
Cycle detection	$\Theta(V + E)$	$\Theta(V)$	a back edge, or none
Articulation vertices + bridges	$\Theta(V + E)$	$\Theta(V)$	cut vertices, cut edges
Topological sort (DFS or Kahn)	$\Theta(V + E)$	$\Theta(V)$	linear order, or "cyclic"
DAG path counting / DP	$\Theta(V + E)$	$\Theta(V)$	counts / optima
SCC (Kosaraju)	$\Theta(V + E)$, 2 passes + transpose	$\Theta(V + E)$	component id per vertex
SCC (Tarjan)	$\Theta(V + E)$, 1 pass	$\Theta(V)$	component id per vertex
Transpose G^T	$\Theta(V + E)$	$\Theta(V + E)$	reversed graph

One-Page Recall

- **Model first.** Directed? Weighted? Simple? Sparse? Cyclic? Embedded? Implicit? Labeled? The flavor decides the representation and the algorithm.
- **Adjacency lists, $\Theta(V+E)$ memory, for almost everything.** Matrices only for dense graphs or when you need $O(1)$ edge tests. Manhattan: 3 000 vertices, 6

000 edges, 9 000 000 matrix entries.

- **Three vertex states:** undiscovered → discovered (on the frontier) → processed. Mark on discovery, never expand twice. Each undirected edge is seen twice, each directed edge once.
- **BFS = queue.** `Θ(V+E)`. `v.d = δ(s,v)`, and the tree path is a shortest path. Queue `d` values are always `⟨k,…,k,k+1,…,k+1⟩`. **Unweighted only.**
- **DFS = stack/recursion.** `Θ(V+E)`. Timestamps `u.d < u.f`; a forest, not a tree.
- **Parenthesis theorem:** intervals are disjoint or properly nested; nested ⟺ descendant.
- **White-path theorem:** `v` is a descendant of `u` iff a white path `u ⇝ v` exists at time `u.d`. **This proves everything else.**
- **Edge classes from the target's color when first explored:** WHITE = tree, GRAY = back, BLACK = forward (`u.d<v.d`) or cross (`u.d>v.d`). **Undirected graphs have only tree and back edges.**
- **The parent guard** (`v != parent[u]`) is what stops an undirected edge's second traversal from being read as a back edge. Skip it and every graph looks cyclic.
- **Cycle ⟺ back edge.** Directed graph acyclic ⟺ DFS finds no back edges (Lemma 20.11).
- **Topological sort = reverse finish order** (`Θ(V+E)`, three lines) or **Kahn's in-degree-0 removal**. Topological order is the DP evaluation order on a DAG.
- **Articulation vertices with `low[]`:** back edges are "security cables". Non-root `v` is a cut vertex iff some child `c` has `low[c] ≥ disc[v]`; root is one iff it has ⩾ 2 children; `(v,c)` is a **bridge** iff `low[c] > disc[v]`. Relax back edges with `disc[v]`, not `low[v]`.
- **SCC (Kosaraju):** DFS `G` for finish times → build `Gᵀ` → DFS `Gᵀ` in **decreasing finish order** → each tree is one SCC. Works because `f(C) > f(C′)` whenever `C → C′`, so in `Gᵀ` each root's component is a sink you can't escape.
- **The condensation `G^SCC` is always a DAG.** Kosaraju numbers components in topological order; Tarjan in reverse.
- **`Θ(V+E)` is optimal** — *"this is as fast as one can ever hope to just read an `n`-vertex, `m`-edge graph."*

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
BFS on an implicit grid graph	200 · Number of Islands	the graph is never built — the grid *is* the adjacency list
BFS = shortest path when unweighted	[1091? use] 847 · Shortest Path Visiting All Nodes	BFS over a **state** graph `(node, visitedMask)`; the layering argument still holds
Two-colouring	785 · Is Graph Bipartite?	BFS or DFS with a colour array; an odd cycle is the only obstruction
Copy a graph by traversal	133 · Clone Graph	forces you to be explicit about the visited map
Cycle detection in a digraph	207 · Course Schedule	a **back edge** ⟺ a cycle; that is `A2`'s edge classification, used once
Topological sort	210 · Course Schedule II	`A3` verbatim — do it once with DFS finish times and once with Kahn
Bridges (Tarjan's `low[]`)	1192 · Critical Connections in a Network	the articulation/bridge machinery of §5, and the only place it appears on LeetCode
DP over a DAG	329 · Longest Increasing Path in a Matrix	memoised DFS; the subproblem graph *is* a DAG (M11)
Components under a query stream	547 · Number of Provinces	DFS answer vs the union-find answer of M10 — write both, then argue

Beyond LeetCode. CSES Problem Set — the *Graph Algorithms* section is the best structured graph ladder available. Codeforces `graphs` tag · `dfs and similar` tag.

The drill that matters here: for every graph problem, say out loud *"directed or undirected? weighted or not? is it a DAG?"* before choosing anything. Those three questions eliminate most of the algorithm catalogue, and getting them wrong is how people end up running Dijkstra on an unweighted graph or a topological sort on something with a cycle.

C++ Toolkit for This Module

Language material from Weiss, ***Data Structures and Algorithm Analysis in C++****, 4th ed., ch. 9 and §3.6–3.7.*

1. `queue` for BFS, explicit stack (or recursion) for DFS

```
void traversalContainers() {
    queue<int> q;      // FIFO -- push()/front()/pop(). BFS.
    stack<int> s;      // LIFO -- push()/top()/pop().   DFS.
    // Note the asymmetry that trips everyone up: queue has
front(), stack has
    // top(). And NEITHER pop() returns the element -- you must
read it first.
    (void)q; (void)s;
}
```

`std::queue` and `std::stack` are **container adaptors**: they wrap a `deque` by default and expose only the operations the abstraction allows. That restriction is the point — you cannot accidentally index into a queue.

Swap the queue for a stack in BFS and you do *not* get DFS. You get a traversal that visits the same vertices in a different order but computes wrong `d` values, because the "mark on enqueue" discipline that makes BFS's layering work is specific to FIFO order. Real DFS marks on *discovery* and uses the recursion (or an explicit stack with an iterator per frame, as in `A2`).

2. Recursion depth, again — and it is worse here

DFS recurses to the depth of the DFS tree, which on a path-shaped graph is `V`. At `V` ≈ 10^5–10^6 that overflows the default 8 MB stack and **segfaults with no message**. This is not exotic: "a linked list of `n` nodes" and "a grid where every cell connects to the next" are both path graphs. `A2` gives the explicit-stack version for exactly this reason.

3. `vector<char>` for colours, not `vector<bool>`

```
enum Color { White, Gray, Black };
void colourArrays(int n) {
    vector<char> color(n, White);   // one byte each, ordinary
references
    // vector<bool> would be bit-packed with a proxy operator[]
(M07 toolkit 7)
    (void)color;
}
```

Three states genuinely need three values. **A single `visited` boolean is enough for BFS and for connectivity, but not for cycle detection**: distinguishing "in progress" (grey) from "finished" (black) is exactly what makes a back edge detectable. Collapsing the three colours into one bool is the standard reason a cycle detector reports false positives on a DAG with diamonds.

4. Building the reverse graph

SCC needs G^T. Build it in one pass, and note that this is $\Theta(V+E)$ — it does not change the asymptotic cost of Kosaraju:

```
vector<vector<int>> reverseGraph(const vector<vector<int>>& adj) {
    vector<vector<int>> rev(adj.size());
    for (size_t u = 0; u < adj.size(); ++u)
        for (int v : adj[u]) rev[v].push_back((int)u);
    return rev;      // return-by-value: moved, not copied [Weiss 1.5.4]
}
```

With an **adjacency matrix** the reverse is the transpose and you can often skip building it entirely by reading `A[j][i]` instead of `A[i][j]` — one of the few places the matrix representation wins.

5. `reserve` on adjacency lists

`vector<vector<int>> adj(n)` then `adj[u].push_back(v)` reallocates each list $O(\lg \deg)$ times. If you know the degrees (a two-pass count), `adj[u].reserve(deg[u])` removes all of it. For $E = 10^6$ this is a measurable win, and it is the graph-shaped instance of M09's advice.

6. Structured bindings over edges

```
void edgeLoop(const vector<pair<int,int>>& edges) {
    for (const auto& [u, v] : edges) (void)(u + v);   // named, not .first/.second
}
```

`const auto&` avoids copying each pair; `[u, v]` names them. In a module full of `(u,v)` pairs this is the difference between readable and not.

Appendix — C++ for Every Pseudocode Block

```
// The representation every entry below uses: adjacency lists in a
vector of
// vectors. Vertices are 0..n-1.
//
// WHY LISTS, NOT A MATRIX: memory is Theta(V+E) rather than
Theta(V^2), and
// "for each neighbour of u" is Theta(deg u) rather than Theta(V).
Every
// algorithm here is Theta(V+E) with lists and Theta(V^2) with a
matrix. Use a
// matrix only when the graph is dense, or when you need O(1) "is
(u,v) an
// edge?" (see Floyd-Warshall in M15).
struct AdjGraph {
    int n = 0;
    vector<vector<int>> adj;
    explicit AdjGraph(int n_) : n(n_), adj(n_) {}
    void addDirected(int u, int v)   { adj[u].push_back(v); }
    void addUndirected(int u, int v) { adj[u].push_back(v);
adj[v].push_back(u); }
};

// Three colours, not one boolean (toolkit 3).
enum class VColor { White, Gray, Black };
```

A1 BFS

Pseudocode: §3, "The algorithm".

```
struct BfsOutput {
    vector<int> d;      // d[v] = number of edges on a shortest
path s -> v, -1 if unreachable
    vector<int> pi;     // pi[v] = predecessor in the BFS tree, -1
= NIL
};

BfsOutput bfs(const AdjGraph& g, int s) {
    BfsOutput r;
```

```
    r.d.assign(g.n, -1);                                // 1-2  u.d =
infinity (encoded as -1)
    r.pi.assign(g.n, -1);                               //      u.pi =
NIL
    vector<VColor> color(g.n, VColor::White);           //
u.color = WHITE

    color[s] = VColor::Gray;                            // 5  s.color
= GRAY
    r.d[s] = 0;                                         //    s.d = 0
    queue<int> Q;                                       // 8  Q =
empty
    Q.push(s);                                          //
ENQUEUE(Q, s)

    while (!Q.empty()) {                                // 10
        int u = Q.front(); Q.pop();                     // 11 u =
DEQUEUE(Q)
        for (int v : g.adj[u]) {                        // 12 for each
v in Adj[u]
            if (color[v] == VColor::White) {            // 13
                color[v] = VColor::Gray;                // 14
                r.d[v] = r.d[u] + 1;
                r.pi[v] = u;
                Q.push(v);                              // 17
ENQUEUE(Q, v)
                // MARK ON ENQUEUE, not on dequeue. If you mark
when you dequeue,
                // a vertex reachable from two frontier vertices is
enqueued
                // twice, and the queue can grow to Theta(E).
Correctness
                // survives; the complexity does not.
            }
        }
        color[u] = VColor::Black;                       // 18 u.color
= BLACK
    }
    return r;
}

// Recover a shortest path by walking pi backwards, then reversing.
```

```
vector<int> bfsPath(const BfsOutput& r, int s, int v) {
    if (r.d[v] < 0) return {};                          // unreachable
    vector<int> path;
    for (int x = v; x != -1; x = r.pi[x]) path.push_back(x);
    reverse(path.begin(), path.end());
    return path.front() == s ? path : vector<int>{};
}
```

Complexity. $\Theta(V + E)$. Each vertex is enqueued and dequeued exactly once ($\Theta(V)$), and each adjacency list is scanned exactly once when its vertex is dequeued ($\Theta(E)$ total). **Space** $\Theta(V)$ — the queue holds at most one BFS "layer", which can be $\Theta(V)$.

The theory, compressed. Let $\delta(s,v)$ be the true minimum edge count.

- **Lemma 20.1:** $\delta(s,v) \le \delta(s,u) + 1$ for every edge (u,v).
- **Lemma 20.2:** $v.d \ge \delta(s,v)$ always — d is an upper bound.
- **Lemma 20.3:** the queue's d values are non-decreasing and differ by at most 1 — **the queue holds at most two adjacent layers at any moment.** That is the structural fact that makes everything else work.
- **Theorem 20.5:** BFS discovers exactly the reachable vertices, and $v.d = \delta(s,v)$ for all of them.

BFS solves unweighted shortest paths and nothing more. The moment edges have differing weights, the layer argument collapses and you need Dijkstra (M15) — except for weights in $\{0,1\}$, where a deque restores it (0-1 BFS).

A2 DFS and DFS-VISIT

Pseudocode: §4, "The algorithm and its timestamps".

```
struct DfsOutput {
    vector<int> d, f;       // discovery and finish times, 1..2V
    vector<int> pi;
    vector<int> finishOrder;   // vertices in increasing finish
time
};

// The recursive form -- CLRS's, line for line. Depth is O(V)
(toolkit 2).
class DfsRunner {
```

```
public:
    explicit DfsRunner(const AdjGraph& g) : g_(g) {}

    DfsOutput run() {
        out_.d.assign(g_.n, 0);
        out_.f.assign(g_.n, 0);
        out_.pi.assign(g_.n, -1);                    // 1-3
u.color = WHITE, u.pi = NIL
        color_.assign(g_.n, VColor::White);
        time_ = 0;                                   // 4  time = 0
        for (int u = 0; u < g_.n; ++u)               // 5  for each
u in G.V
            if (color_[u] == VColor::White)          // 6      if
u.color == WHITE
                visit(u);                            // 7
DFS-VISIT(G, u)
        return out_;
    }
private:
    void visit(int u) {
        out_.d[u] = ++time_;                         // 1-2  time =
time+1; u.d = time
        color_[u] = VColor::Gray;                    // 3  u.color
= GRAY
        for (int v : g_.adj[u]) {                    // 4  for each
v in Adj[u]
            if (color_[v] == VColor::White) {        // 5
                out_.pi[v] = u;                      // 6
                visit(v);                            // 7
            }
        }
        out_.f[u] = ++time_;                         // 8-9  time =
time+1; u.f = time
        color_[u] = VColor::Black;                   // 10
        out_.finishOrder.push_back(u);               // the order
topological sort wants
    }
    const AdjGraph& g_;
    DfsOutput out_;
    vector<VColor> color_;
    int time_ = 0;
};
```

```cpp
// ITERATIVE DFS with an explicit stack. Necessary above ~10^5 vertices.
//
// The subtlety: a recursive DFS finishes a vertex when its loop over neighbours
// ENDS, so an explicit stack must remember HOW FAR THROUGH that loop each frame
// is. `iter[u]` is that per-frame iterator; without it you cannot compute
// correct finish times, and finish times are the whole point of DFS.
DfsOutput dfsIterative(const AdjGraph& g) {
    DfsOutput out;
    out.d.assign(g.n, 0);
    out.f.assign(g.n, 0);
    out.pi.assign(g.n, -1);
    vector<VColor> color(g.n, VColor::White);
    vector<size_t> iter(g.n, 0);                    // next neighbour index per vertex
    int time = 0;

    for (int s = 0; s < g.n; ++s) {
        if (color[s] != VColor::White) continue;
        vector<int> stk{s};
        color[s] = VColor::Gray;
        out.d[s] = ++time;
        while (!stk.empty()) {
            int u = stk.back();
            if (iter[u] < g.adj[u].size()) {
                int v = g.adj[u][iter[u]++];        // advance THIS frame's iterator
                if (color[v] == VColor::White) {
                    color[v] = VColor::Gray;
                    out.d[v] = ++time;
                    out.pi[v] = u;
                    stk.push_back(v);
                }
            } else {                                 // neighbours exhausted: finish u
                out.f[u] = ++time;
                color[u] = VColor::Black;
```

```
                out.finishOrder.push_back(u);
                stk.pop_back();
            }
        }
    }
    return out;
}
```

Complexity. $\Theta(V + E)$. Space $\Theta(V)$ for the arrays plus $O(V)$ stack.

The two theorems that make DFS more than a traversal.

- **Theorem 20.7 (parenthesis theorem).** For any `u, v`, exactly one of: the intervals `[u.d, u.f]` and `[v.d, v.f]` are **disjoint** and neither is a descendant of the other; `[u.d, u.f]` is **contained in** `[v.d, v.f]` and `u` is a descendant of `v`; or the reverse. **They never partially overlap.** So the discovery/finish times are a correctly nested parenthesisation, and ancestry is an interval-containment test.
- **Theorem 20.9 (white-path theorem).** `v` is a descendant of `u` in the DFS forest **iff** at the moment `u` is discovered, there is a path from `u` to `v` consisting entirely of white vertices.

Edge classification (Theorem 20.10) falls straight out of the colours, and is the whole reason to bother with three of them:

WHEN YOU LOOK AT `(u,v)` AND `v` IS…	EDGE TYPE
white	**tree edge**
grey	**back edge** — `v` is an ancestor, so **this is a cycle**
black, `u.d < v.d`	forward edge
black, `u.d > v.d`	cross edge

A directed graph is acyclic iff DFS finds no back edge. That is the cycle test in "Course Schedule", and it is the entire content of Lemma 20.11. In an **undirected** graph, DFS produces only tree and back edges — no forward or cross edges ever.

A3 TOPOLOGICAL-SORT

Pseudocode: §6, "Topological sort".

```
// DFS-based: run DFS, then reverse the finish order.
// Returns {} if the graph has a cycle (no topological order
exists).
vector<int> topologicalSortDFS(const AdjGraph& g) {
    DfsOutput r = dfsIterative(g);
    // 2  "as each vertex is finished, insert it onto the FRONT of
a list"
    //    -- which is the same as collecting finishes and
reversing.
    vector<int> order(r.finishOrder.rbegin(),
r.finishOrder.rend());

    // The pseudocode assumes a DAG. Verify it, because a
"topological order" of
    // a cyclic graph is silently meaningless: check that every
edge goes
    // forwards in the order produced.
    vector<int> pos(g.n);
    for (int i = 0; i < g.n; ++i) pos[order[i]] = i;
    for (int u = 0; u < g.n; ++u)
        for (int v : g.adj[u])
            if (pos[u] > pos[v]) return {};          // a back
edge: cycle
    return order;
}

// KAHN'S ALGORITHM: repeatedly emit a vertex of in-degree 0.
// Same Theta(V+E), no recursion, and it DETECTS the cycle for free
-- if fewer
// than n vertices are emitted, the rest are stuck in a cycle. This
is the one
// to write in an interview.
vector<int> topologicalSortKahn(const AdjGraph& g) {
    vector<int> indeg(g.n, 0);
    for (int u = 0; u < g.n; ++u)
        for (int v : g.adj[u]) ++indeg[v];

    queue<int> q;
    for (int v = 0; v < g.n; ++v) if (indeg[v] == 0) q.push(v);
```

```
    vector<int> order;
    order.reserve(g.n);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g.adj[u])
            if (--indeg[v] == 0) q.push(v);          // u was v's
last prerequisite
    }
    return (int)order.size() == g.n ? order : vector<int>{};   //
short = cycle
}
```

Complexity. **`O(V + E)` for both.** Space `O(V)`.

Theorem 20.12: `TOPOLOGICAL-SORT` produces a valid topological order of a DAG. *Proof sketch:* for any edge `(u,v)`, when it is explored `v` cannot be grey (that would be a back edge, contradicting acyclicity), so `v` is either white — and finishes before `u` — or already black, hence also `v.f < u.f`. **In a DAG, every edge `(u,v)` has `v.f < u.f`**, so decreasing finish time is a topological order.

Kahn's version also answers a question DFS's does not: *is the order unique?* If the queue ever holds two or more vertices at once, there are multiple valid orders.

A4 STRONGLY-CONNECTED-COMPONENTS

Pseudocode: §7, *"Strongly connected components"*.

```
// KOSARAJU: two DFS passes and one graph reversal.
// Returns comp[v] = component id, numbered in reverse topological
order of the
// condensation (so every edge between components goes from a LOWER
id to a
// HIGHER one -- a property worth having, and free).
vector<int> sccKosaraju(const AdjGraph& g) {
    // 1  DFS on G to compute finish times
    DfsOutput first = dfsIterative(g);

    // 2  create G^T
    AdjGraph gt(g.n);
    for (int u = 0; u < g.n; ++u)
```

```
        for (int v : g.adj[u]) gt.addDirected(v, u);

    // 3  DFS on G^T, considering vertices in order of DECREASING
finish time
    vector<int> comp(g.n, -1);
    int c = 0;
    for (auto it = first.finishOrder.rbegin(); it !=
first.finishOrder.rend(); ++it) {
        if (comp[*it] != -1) continue;
        // Iterative flood fill: each tree of this second forest is
one SCC.
        vector<int> stk{*it};
        comp[*it] = c;
        while (!stk.empty()) {
            int u = stk.back(); stk.pop_back();
            for (int v : gt.adj[u])
                if (comp[v] == -1) { comp[v] = c; stk.push_back(v);
}
        }
        ++c;                                            // 4  one
tree = one SCC
    }
    return comp;
}

// TARJAN: ONE pass, no reversal. Faster in practice and the usual
choice.
//
// low[u] = the smallest index reachable from u's subtree using
tree edges plus
// AT MOST ONE back edge. When low[u] == index[u], u is the ROOT of
an SCC, and
// everything above u on the stack is that component.
vector<int> sccTarjan(const AdjGraph& g) {
    vector<int> index(g.n, -1), low(g.n, 0), comp(g.n, -1), stk;
    vector<char> onStack(g.n, 0);
    vector<size_t> iter(g.n, 0);
    int counter = 0, c = 0;

    for (int s = 0; s < g.n; ++s) {
        if (index[s] != -1) continue;
```

```
        vector<int> call{s};                            // explicit
call stack (toolkit 2)
        index[s] = low[s] = counter++;
        stk.push_back(s); onStack[s] = 1;
        while (!call.empty()) {
            int u = call.back();
            if (iter[u] < g.adj[u].size()) {
                int v = g.adj[u][iter[u]++];
                if (index[v] == -1) {                   // tree edge:
descend
                    index[v] = low[v] = counter++;
                    stk.push_back(v); onStack[v] = 1;
                    call.push_back(v);
                } else if (onStack[v]) {                // back/cross
edge INSIDE the
                    low[u] = min(low[u], index[v]);     // current
component
                    // `index[v]`, not `low[v]`: using low here is
the classic
                    // Tarjan bug. It happens to work for SCC but
breaks the
                    // same skeleton when reused for bridges.
                }
            } else {
                call.pop_back();
                if (!call.empty()) low[call.back()] =
min(low[call.back()], low[u]);
                if (low[u] == index[u]) {               // u roots an
SCC
                    for (;;) {
                        int w = stk.back(); stk.pop_back();
onStack[w] = 0;
                        comp[w] = c;
                        if (w == u) break;
                    }
                    ++c;
                }
            }
        }
    }
    // Tarjan emits components in REVERSE topological order; flip
the ids so the
```

```
    // convention matches Kosaraju's above.
    for (int& x : comp) x = c - 1 - x;
    return comp;
}

// The CONDENSATION: contract each SCC to a single vertex. The
result is ALWAYS
// a DAG -- if it had a cycle, all the components on that cycle
would be
// mutually reachable and would therefore be one component, a
contradiction.
AdjGraph condensation(const AdjGraph& g, const vector<int>& comp) {
    int c = 0;
    for (int x : comp) c = max(c, x + 1);
    AdjGraph dag(c);
    set<pair<int,int>> seen;                          // de-
duplicate parallel edges
    for (int u = 0; u < g.n; ++u)
        for (int v : g.adj[u])
            if (comp[u] != comp[v] && seen.insert({comp[u],
comp[v]}).second)
                dag.addDirected(comp[u], comp[v]);
    return dag;
}
```

Complexity. $\Theta(V + E)$ for both. Kosaraju does two DFS passes plus a reversal — three $\Theta(V+E)$ sweeps and $\Theta(V+E)$ extra memory for G^T. Tarjan does **one** pass and no reversal, which is why it wins in practice despite being harder to write.

Why Kosaraju works (Lemmas 20.13–20.15, Theorem 20.16). The key lemma: if C and C' are distinct SCCs and there is an edge from C to C', then $\max\{u.f : u \in C\} > \max\{u.f : u \in C'\}$. So processing vertices in **decreasing finish order** visits the components in topological order of the condensation — and in G^T all the *outgoing* edges of a component now point at components already assigned, so the flood fill cannot escape the current SCC.

Why you care. The condensation turns any directed graph into a DAG, and on a DAG you get topological sort, DP over vertices, longest paths, and 2-SAT. **"Find the SCCs, then work on the condensation" is a standard first move on any hard directed-graph problem.**

Next: M14 — Minimum Spanning Trees (CLRS 21 + Skiena 8.1) — the cut property, Kruskal with union–find, Prim with a priority queue, and why both are the same greedy algorithm in different clothes.